

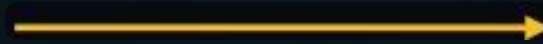
```
$ ./lua-arena --script inimigo.lua
```

LuaArena

Lua como Linguagem de Extensão para Aplicações

Integrantes: David Josué, Carlos Eduardo, Levi Farias, Henrique Coimbra, Jetro Kepler, Ângelo Gabriel

C++ HOST



LUA SCRIPT

O problema que motivou o uso de Lua

Regras mutáveis não precisam ficar presas ao binário compilado.

Código C++ fixo

Quando regras de comportamento ficam embutidas no C++, toda mudança exige editar fonte, recompilar e redistribuir.

Recompilação recorrente

Ajustes de balanceamento, regras de inimigos ou configuração viram alterações no núcleo, mesmo quando são apenas políticas.

Objetivo do LuaArena: demonstrar extensão dinâmica real sem recompilar o motor C++.

Extensão dinâmica

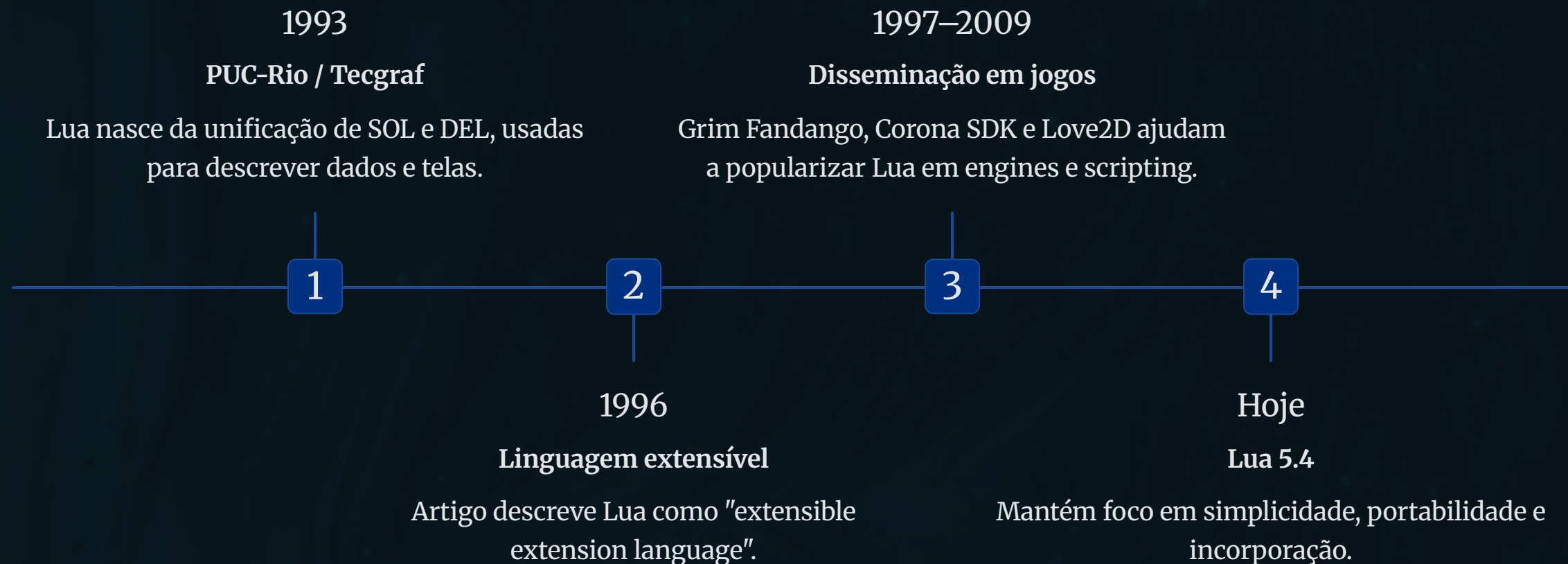
A solução estudada move essas decisões para scripts Lua carregados em tempo de execução, mantendo o motor estável.

C++ com regras fixas → recompilar
→ C++ estável + Lua scripts



Contexto histórico da Lua

Origem brasileira e objetivo inicial de extensão/configuração.



Lua nasceu para ser pequena, portátil e fácil de embutir — exatamente o cenário estudado no LuaArena.

Lua como linguagem de extensão

O programa hospedeiro oferece pontos controlados de personalização.

C++ — linguagem hospedeira

- implementa o programa principal e o loop
- controla vida, energia, turnos e vitória/derrota
- define capacidades expostas aos scripts
- valida retornos antes de aplicar efeitos

Lua — linguagem de extensão

- descreve habilidades, inimigos, arenas e configuração
- opera dentro da interface oferecida pelo hospedeiro
- recebe apenas dados selecionados
- não é proprietária do estado real do jogo

Lua decide ou descreve. C++ valida e aplica.

Conceitos fundamentais da integração

Os mecanismos que tornam a incorporação tecnicamente viável.

Embedding

Incorporar o runtime Lua ao processo C++. O motor cria e controla um `lua_State`, sem iniciar processo externo.

Binding

Adaptar dados e funções: tabelas Lua representam visões temporárias de jogador/inimigo; funções C++ podem ser expostas.

Runtime / VM / bytecode

Lua carrega chunks, gera uma representação interna e executa a lógica por sua máquina virtual sob controle do hospedeiro.

Stack da API C

A comunicação usa uma pilha virtual para argumentos, retornos e erros; índices e tipos exigem disciplina.



Manifestação no LuaArena

Conceito	No projeto	Autoridade
Hospedeiro	executável e loop da batalha	C++
Extensão	habilidades, inimigos, arenas e configuração	Lua
Binding	tabelas de personagens, retornos e game_log	conversão
Fronteira	validação antes de aplicar qualquer resultado	C++

Características da linguagem Lua

Pequena por desenho, flexível por mecanismos simples.

Múltiplos paradigmas

Lua oferece um conjunto pequeno de recursos gerais que podem ser combinados em estilos diferentes, em vez de impor um modelo rígido.

Tabelas como base

A tabela é a estrutura central: representa arrays, dicionários, registros e objetos simulados com metatables.

Funções de 1ª classe

Funções podem ser passadas, armazenadas e redefinidas; closures permitem comportamento parametrizado.

Retornos múltiplos e varargs

Funções podem retornar vários valores e receber quantidade variável de argumentos, útil para APIs enxutas.

```
function makeaddfunc(x)
  return function(y)
    return x + y
  end
end
```

```
plustwo = makeaddfunc(2)
print(plustwo(5)) -- 7
```

Mecanismos ligados ao projeto

- metatables e metamétodos permitem especializar comportamento
- coletor de lixo gerencia objetos Lua e closures
- tipagem dinâmica aumenta flexibilidade, mas exige validação
- sintaxe simples favorece scripts curtos de configuração

Interoperabilidade C++ ↔ Lua no LuaArena

Chamada protegida, stack e conversão de contratos.



Interoperabilidade aqui não é comunicação por rede: Lua roda incorporada ao mesmo processo C++.

Fronteira de confiança e arquitetura

A flexibilidade do script só é segura quando existe contrato.

```
ActionResult {  
  tipo, valor, mensagem,  
  efeito, duracao, custo  
}
```

Por que validar?

Mesmo scripts do repositório podem conter erros, campos ausentes, tipos incorretos ou valores fora do domínio. Dados vindos de Lua são entradas não confiáveis até que o motor valide.

Lua pode

- descrever uma ação
- calcular valores dentro do contrato
- usar dados fornecidos pelo C++

C++ precisa

- conferir tipos e campos obrigatórios
- rejeitar efeitos inválidos
- preservar estado diante de falhas

O benefício não é só editar scripts; é criar uma fronteira pequena, verificável e documentada.

LuaArena como exemplo prático

O motor C++ mantém estado; Lua varia políticas e conteúdo.

```
./build/luarena scripts/enemies/goblin_basic.lua --arena scripts/arenas/volcanic.lua --difficulty scripts/difficulty/normal.lua
```

Motor C++

vida, energia, ataque, defesa, turnos e resultado da batalha.

Scripts Lua

inimigos, habilidades, arenas, dificuldade e eventos.

Hooks

usar_habilidade, escolher_acao e eventos de ciclo de vida.

Efeitos

cura, queimadura, veneno e alterações controladas pelo motor.

Mesmo executável. Script diferente. Comportamento diferente.

Demonstração da extensão dinâmica

O comportamento muda trocando apenas o script Lua.

```
$ ./build/luarena scripts/enemies/goblin_basic.lua
```

```
# inimigo usa ataque-base
```

```
$ ./build/luarena scripts/enemies/goblin_aggressive.lua
```

```
# se jogador < 50% de vida, usa 1.5x do ataque
```

```
function escolher_acao(inimigo, jogador)
    if jogador.vida < jogador.vida_max * 0.5 then
        return { tipo = "ataque",
                 valor = inimigo.ataque * 1.5 }
    end

    return { tipo = "ataque",
            valor = inimigo.ataque }
end
```

Conclusão da demo

A política do inimigo muda sem recompilar o C++: Lua descreve a decisão, C++ valida e aplica.

Aplicações reais de Lua

A adoção se concentra em scripting, extensão e configuração.



Jogos e engines

Grim Fandango, World of Warcraft, Love2D, Solar2D/Corona e Roblox: regras, UI e comportamento ajustável.



Ferramentas extensíveis

Nmap e Wireshark usam Lua para scripts, plugins e análise configurável.



Sistemas embarcados

OpenWrt e roteadores exploram runtime pequeno e configuração programável.



Automação e configuração

Quando o núcleo deve permanecer estável, scripts permitem lógica sem recompilar.

Padrão comum: núcleo estável + políticas alteráveis por script.



Lua × Python como linguagens embarcadas

Comparação focada em incorporação em aplicações C/C++.

Critério	Lua 5.4	CPython 3	Leitura prática
API	stack virtual	PyObject*, refs e exceções	ambas viáveis; modelos diferentes
Runtime	núcleo pequeno e libs selecionáveis	runtime amplo + biblioteca-padrão	Lua é mais enxuta no escopo restrito
Distribuição	pode ser ligada ao executável	envolve CPython e caminhos/módulos	Python exige mais configuração
Segurança	sem sandbox automática	sem sandbox automática	host precisa controlar capacidades
LuaArena	contratos simples em tabelas	viável, mas pesado para o objetivo	Lua encaixa melhor no protótipo

A conclusão não é "Lua é sempre melhor": para o LuaArena, a fronteira pequena favorece Lua.

Análise crítica: maturidade, ecossistema e adoção

Lua é madura, mas seu ecossistema é propositalmente mais concentrado.

Maturidade

Existe desde 1993, com evolução estável e manutenção pelo grupo original. Limitação: LuaJIT permanece ligado à linha 5.1.

Ecossistema e ferramentas

Biblioteca-padrão pequena; LuaRocks é o gerenciador mais usado, mas não oficial; ferramentas existem, porém menos padronizadas.

Comunidade

Ativa e concentrada em jogos, addons, scripting, sistemas embarcados e configuração — forte em nichos, menor fora deles.

Perspectiva realista

Lua não tende a virar linguagem de propósito geral dominante. Sua adoção permanece forte no nicho para o qual foi desenhada: extensão embutida, leve e previsível.

Leitura para o LuaArena

A escolha é madura e adequada para scripts curtos de batalha, desde que o projeto valide retornos e mantenha contratos explícitos.

Vantagens e limitações no projeto

O ganho vem da fronteira bem desenhada entre host e script.

Vantagens

- baixo acoplamento entre motor e conteúdo
- alteração de comportamento sem recompilar
- API C pequena e explícita
- boa portabilidade e distribuição enxuta
- host decide exatamente o que expõe aos scripts

Limitações

- tipagem dinâmica desloca erros para runtime
- stack da API C exige disciplina
- contratos precisam ser validados manualmente
- sandbox não vem pronta
- biblioteca-padrão mínima e ecossistema menor

Boa arquitetura: Lua varia políticas e conteúdo; C++ preserva estado, invariantes e segurança.

Síntese e referências bibliográficas

O protótipo responde ao tema pela combinação entre teoria e implementação.

❏ **Lua estende. C++ mantém o controle.**

No LuaArena, o mesmo executável C++ muda de comportamento ao carregar scripts Lua diferentes. Isso demonstra Lua como linguagem de extensão incorporada a uma aplicação C++.

Referências principais

- [Lua.org](#) — Lua 5.4 Reference Manual; Welcome to Lua 5.4; source code
- Ierusalimschy, Figueiredo & Celes — Lua: An Extensible Extension Language (1996)
- Ierusalimschy, Figueiredo & Celes — The Evolution of Lua (HOPL, 2007)
- [Python.org](#) — Embedding CPython; Python/C API; initialization configuration
- Fowler — Inversion of Control; Parnas — decomposição em módulos



PERGUNTAS?